Lab #4: Odometry
November 2, 2026

Group Members: Andres Permuy (app2164), Howard Wang (hw3214), Samwel Waweru (swk2139)

MANDATORY AI DISCLOSURE:

LaTeX was used since this report needed text, code snippets, figures, and equations. For assignments where equations, calculations, and model templates were taken from the lecture slides and/or homework examples, ChatGPT was used to help quickly convert those into LaTeX equations.

It was also used in general for the following:

- LaTeX code for formatting to make everything fit in the report nicely.
- Grammar check
- Syntax checking
- rosbag help and how-to
- Calculation sanity check

Assignment 1:

The ground-truth body velocities were recorded from `/tb3/ground_truth/twist` with nominally zero commanded velocity. The recorded components were linear velocity v_x (the ROS field `twist.linear.x`) and angular velocity ω_z (the ROS field `twist.angular.z`). These correspond to v and ω, respectively, in the planar odometry state used in the later assignments. The analysis script uses the Python names `vx` and `wz` for these components.

Calculations:

Lecture 1: mean, variance, and standard deviation are defined as:

$$\mu_q = E[q], \qquad V[q] = E\left[(q - E[q])^2\right], \qquad \sigma_q = \sqrt{V[q]}, \tag{1}$$

where q represents either v_x or ω_z.

Given:

The recording spans 35.5 s and contains $N = 1648$ finite samples of each component. The file `analyze_zero_cmd.py` calculates the means using `numpy.mean()` and the *sample* standard deviations using `numpy.std(ddof=1)`, i.e. the estimators

$$\hat{\mu}_q = \frac{1}{N}\sum_{i=1}^{N} q_i, \qquad \hat{\sigma}_q = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(q_i - \hat{\mu}_q)^2}. \tag{2}$$

The bag, script, and results are in `submission/lab4_Odometry/assignment1/` (`data/zero_cmd_bag.zip` and `data/zero_cmd_velocity_summary.txt`).

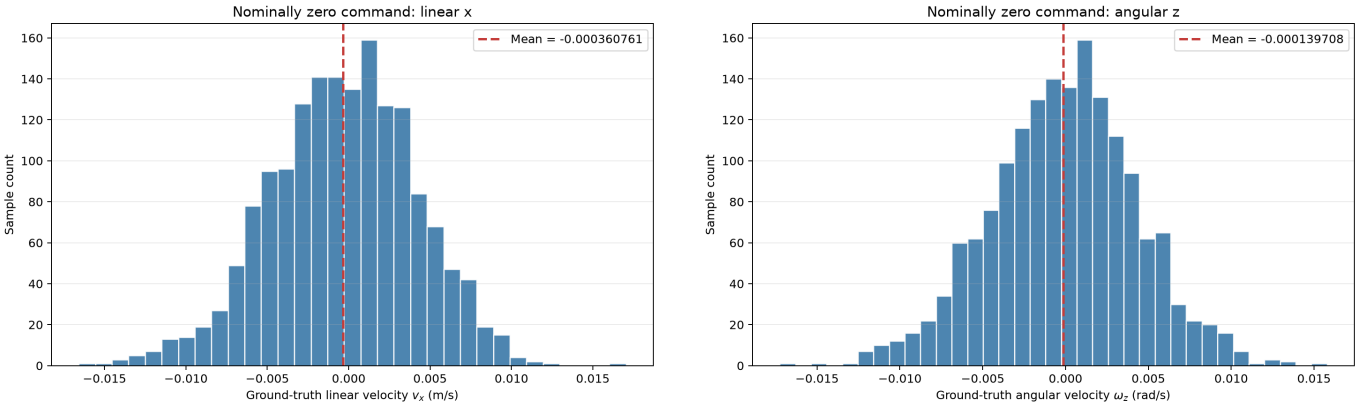
Results:

$$\therefore \boxed{\hat{\mu}_{v_x} = -0.000360761 \text{ m/s}, \qquad \hat{\sigma}_{v_x} = 0.004448870 \text{ m/s}} \tag{3}$$

$$\boxed{\hat{\mu}_{\omega_z} = -0.000139708 \text{ rad/s}, \qquad \hat{\sigma}_{\omega_z} = 0.004391655 \text{ rad/s}} \tag{4}$$

Here, $\hat{\mu}$ and $\hat{\sigma}$ denote values estimated from the recorded samples, rather than population parameters known in advance. Both histograms in Figure 1 show measured velocities concentrated near zero. Both means are below 10% of the corresponding $\hat{\sigma}$, so the zero-command noise is effectively zero-mean.

Source of the noise: `turtlebot3_empty_launch.py` sets `cmd_noise:=0.005`, and the `cmd_vel_noise_injector` node adds $\mathcal{N}(0, 0.005^2)$ to both v and ω of every command it forwards; when no command has arrived for 1 s, it publishes noisy zero commands at 30 Hz. The measured $\hat{\sigma} \approx 0.0044$ is of the same order and slightly below the injected 0.005, consistent with the robot's first-order velocity response (Assignment 3) partially smoothing the injected noise. These variances are used as the input-noise covariance Σ_u in Assignments 4 and 6.



(a) Linear velocity v_x (m/s). (b) Angular velocity ω_z (rad/s).

Figure 1: Ground-truth velocity histograms for nominally zero commanded motion. Dashed red lines indicate the sample means.

Assignment 2:

1. Commanded and measured velocities:

The robot was commanded through `/cmd_vel` using `geometry_msgs/msg/TwistStamped`, with commands published at 10 Hz. Linear and angular velocities were measured using `/tb3/ground_truth/twist` and compared with the commands in PlotJuggler.

$$u_v = 0.15 \text{ m/s}, \qquad u_\omega = 0.40 \text{ rad/s} \tag{5}$$

Results:

The measured velocities v and ω closely followed the respective commanded velocities u_v and u_ω, with small fluctuations visible in both signals (Figure 2).

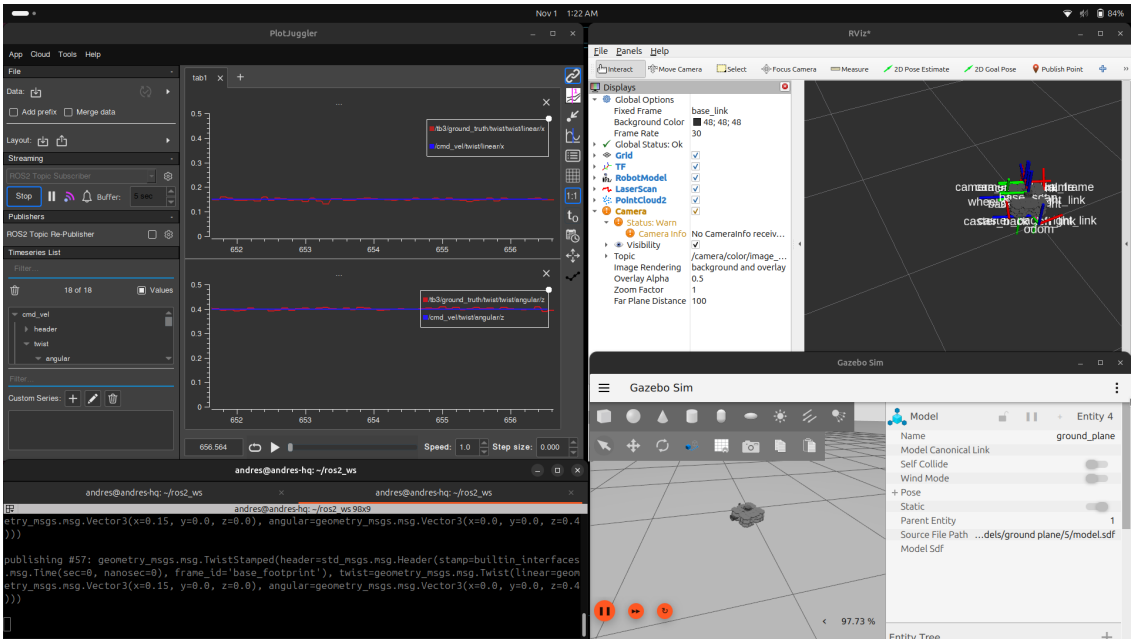
2. Ground-truth pose and reference frame:

The pose topic `/tb3/ground_truth/pose` publishes `geometry_msgs/msg/PoseStamped`. Its message header reports `frame_id: world`, so RViz was set to the `world` fixed frame to display the ground-truth pose.

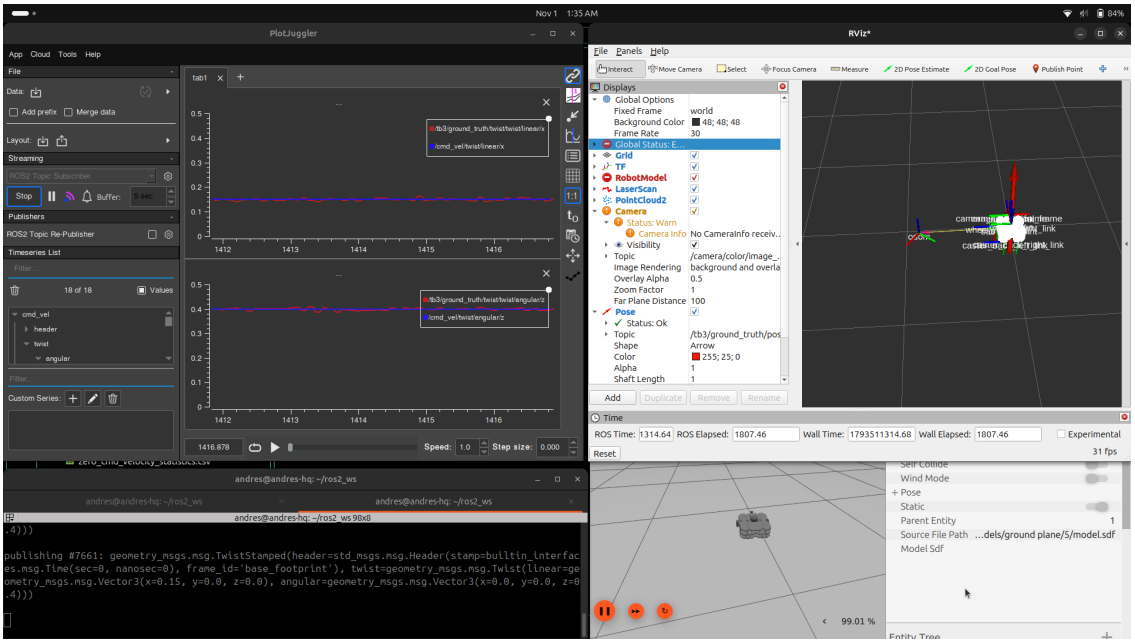
With Fixed Frame = `world`, the pose arrow was drawn, but the RobotModel display reported a TF error, RViz's Global Status changed to Error, and the robot appeared only as a white silhouette (Figure 2b). With a frame from the robot's own TF tree, such as `base_link`, the robot rendered normally and Global Status was OK (Figure 2a), but the `world`-stamped pose could not be displayed. The cause is the TF tree. RViz draws the robot model through TF, and the simulator's DiffDrive plugin publishes only `odom`→`base_footprint` (with `robot_state_publisher` adding the links below it). No node publishes `world`→`odom`, so the robot's tree is disconnected from `world`: RViz cannot place the robot in `world`, and cannot transform the `world`-stamped pose into the robot's frames. Because `turtlebot3_empty_launch.py` spawns the robot at $(0, 0)$ with zero yaw, `odom` coincides with `world` at startup, and the fix is an identity static transform:

```
ros2 run tf2_ros static_transform_publisher --frame-id world --child-frame-id odom
```

The screenshots and videos below were recorded before this fix, so they show the two views separately: the robot in its own frame, and the ground-truth pose in `world` (Figure 2). The same static transform is used in Assignment 7 to overlay both poses.



(a) PlotJuggler: commanded (blue) and ground-truth (red) v (top) and ω (bottom) for $u_v = 0.15\,\text{m/s}$, $u_\omega = 0.40\,\text{rad/s}$. RViz Fixed Frame = `base_link`: robot model renders, Global Status OK.



(b) RViz Fixed Frame = `world`: the ground-truth pose (red arrow) is drawn, but RobotModel and Global Status show errors because no `world`→`odom` transform exists.

Figure 2: TurtleBot 3 motion experiment using PlotJuggler, RViz, and Gazebo.

Video demonstrations: Velocity tracking and robot visualization | Ground-truth pose visualization.

Assignment 3:

The robot was commanded with successive 2 s velocity steps using `/cmd_vel`. The ground-truth responses v and ω were recorded from `/tb3/ground_truth/twist` in a ROS 2 bag along with the commands u_v and u_ω. Each signal was sampled at approximately 50 Hz.

Calculations:

The Lab 4 instructions give the first-order update law and the forgetting factor determined from the 90% rise time:

$$v[n+1] = a_v v[n] + G_v(1-a_v)u_v[n], \tag{6}$$

$$\omega[n+1] = a_\omega \omega[n] + G_\omega(1-a_\omega)u_\omega[n], \tag{7}$$

$$a_v(\Delta t) = 0.1^{\Delta t/\tau_v}, \qquad a_\omega(\Delta t) = 0.1^{\Delta t/\tau_\omega}. \tag{8}$$

Here, G_v and G_ω are the steady-state gains and τ_v and τ_ω are the times to reach 90% of each velocity step.

For each transition, the initial velocity was averaged over a settled interval before the command, and the final velocity was averaged near the end of the 2 s stage. The gain was calculated as the ratio of measured to commanded velocity changes. The 90% point was determined relative to the measured initial and final velocities; its timestamp was interpolated between neighboring recorded samples.

Results:

Velocity	Command step	Gain G	90% rise time τ (s)
v	$0.00 \to 0.03$	1.073	0.036
v	$0.03 \to 0.06$	0.956	0.030
v	$0.06 \to 0.09$	0.959	0.040
v	$0.09 \to 0.12$	1.024	0.039
v	$0.12 \to 0.15$	0.970	0.039
ω	$0.00 \to 0.10$	1.013	0.019
ω	$0.10 \to 0.20$	1.008	0.027
ω	$0.20 \to 0.30$	1.008	0.020
ω	$0.30 \to 0.40$	1.006	0.019

The linear command steps are in m/s; angular command steps are in rad/s. For example, the first linear step gave

$$G_{v,1} = \frac{0.030312 - (-0.001882)}{0.03} = 1.073, \qquad \tau_{v,1} = 6153.536 - 6153.500 \approx 0.036\,\text{s}. \tag{9}$$

The first angular step similarly gave $G_{\omega,1} = 1.013$ and $\tau_{\omega,1} \approx 0.019$ s (Figure 4). Averaging the step estimates gives:

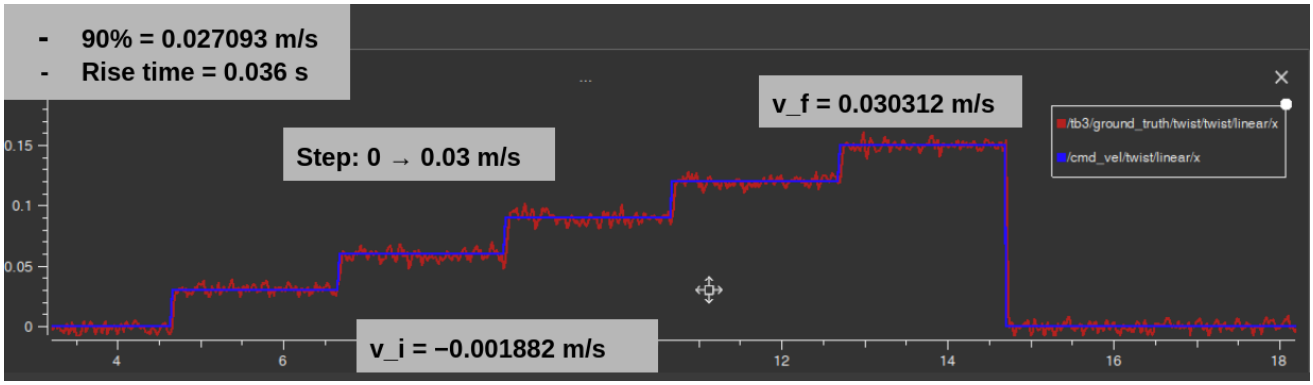
$$\therefore \boxed{G_v \approx 0.9965, \quad \tau_v \approx 0.0367\,\text{s}} \tag{10}$$

$$\boxed{G_\omega \approx 1.0085, \quad \tau_\omega \approx 0.0212\,\text{s}} \tag{11}$$

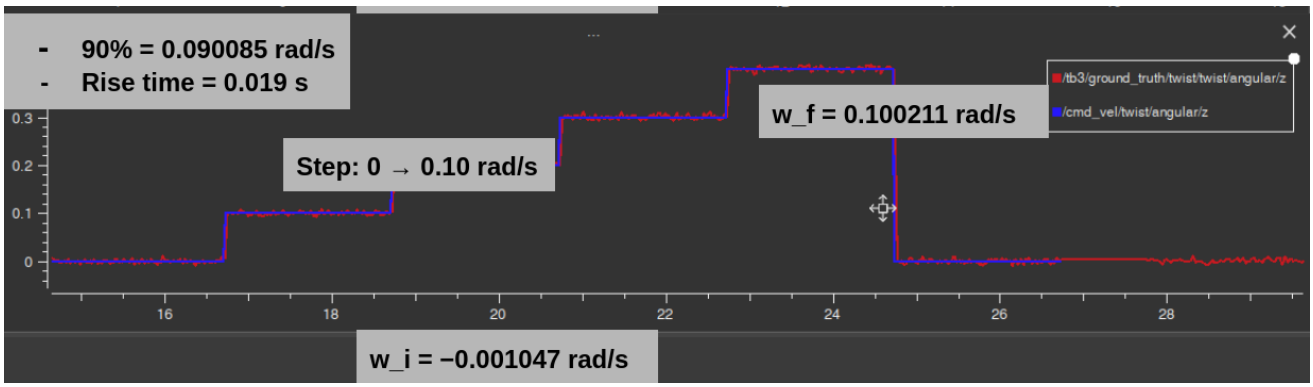
Thus, the experimentally identified forgetting factors are:

$$\boxed{a_v(\Delta t) = 0.1^{\Delta t/0.0367}, \qquad a_\omega(\Delta t) = 0.1^{\Delta t/0.0212}} \tag{12}$$

$$\boxed{a_v(0.020) \approx 0.286, \qquad a_\omega(0.020) \approx 0.114}. \tag{13}$$



(a) Linear velocity steps of 0.03 m/s.



(b) Angular velocity steps of 0.10 rad/s.

Figure 3: Commanded (blue) and measured (red) velocity staircase responses recorded in PlotJuggler. The annotated values refer to the first step, shown in detail in Figure 4.

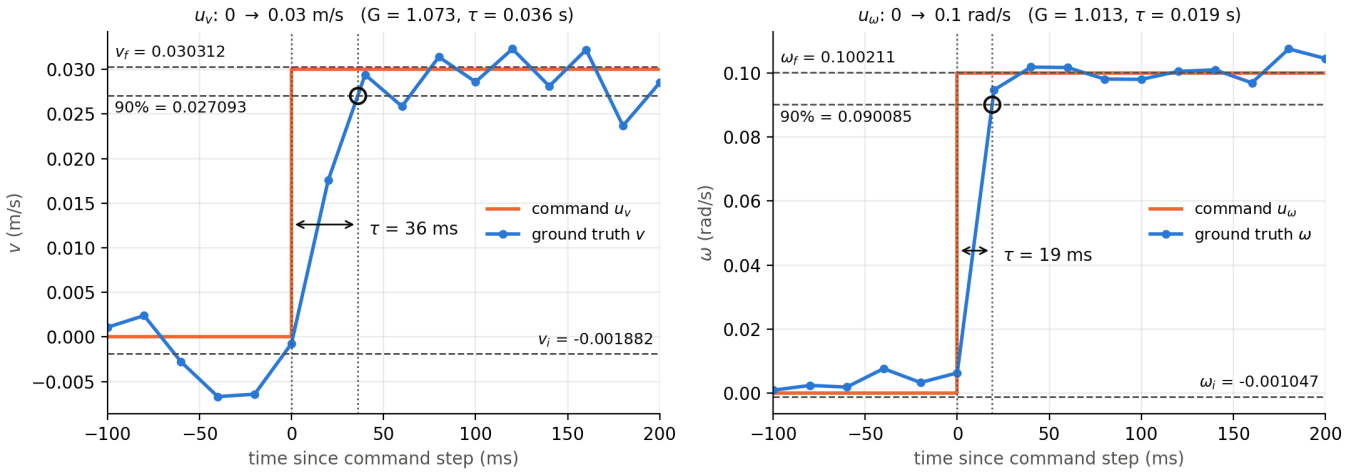


Figure 4: First linear and first angular step, plotted from the recorded bag at the 20 ms sample resolution. Dashed lines: initial, final, and 90% levels; circle: interpolated 90% crossing that defines τ. Generated by `plot_step_zoom.py`.

Assignment 4:

1. State and command:

The state and command vectors are

$$\mathbf{x}[n] = \begin{bmatrix}\theta[n] & x[n] & y[n] & v[n] & \omega[n]\end{bmatrix}^T, \qquad \mathbf{u}[n] = \begin{bmatrix}u_v[n] & u_\omega[n]\end{bmatrix}^T. \tag{14}$$

Here, x and y are positions, θ is heading, and v and ω are body linear and angular velocities. As in the odometry lecture, $v = v_x$ and the lateral body velocity is zero. The time between updates is Δt.

2. Pose update:

Multiplying out the 2D motion product from the Lab 4 instructions, with body-frame displacement $[v\Delta t,\ 0]^T$ (zero lateral velocity),

$$\begin{bmatrix} R(\theta[n+1]) & \mathbf{p}[n+1] \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} R(\theta[n])R(\omega[n]\Delta t) & \mathbf{p}[n] + R(\theta[n]) \begin{bmatrix} v[n]\Delta t \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}. \tag{15}$$

The rotation block gives $R(\theta[n+1]) = R(\theta[n] + \omega[n]\Delta t)$, and the translation block gives the position update:

$$\begin{aligned} \theta[n+1] &= \theta[n] + \omega[n]\Delta t, \\ x[n+1] &= x[n] + v[n]\Delta t \cos\theta[n], \\ y[n+1] &= y[n] + v[n]\Delta t \sin\theta[n]. \end{aligned} \tag{16}$$

3. Velocity dynamics:

Using the first-order model from the Lab 4 instructions (page 3) and the measured parameters from Assignment 3,

$$\begin{aligned} v[n+1] &= a_v v[n] + G_v(1-a_v)u_v[n], \\ \omega[n+1] &= a_\omega \omega[n] + G_\omega(1-a_\omega)u_\omega[n], \end{aligned} \tag{17}$$

where

$$\begin{aligned} a_v = 0.1^{\Delta t/\tau_v}, \qquad a_\omega = 0.1^{\Delta t/\tau_\omega}, \\ G_v = 0.9965, \quad \tau_v = 0.0367\,\text{s}, \qquad G_\omega = 1.0085, \quad \tau_\omega = 0.0212\,\text{s}. \end{aligned} \tag{18}$$

Because Δt is measured from the sensor timestamps at every update, a_v, a_ω, and therefore B and G_x below, are recomputed at each step.

4. Nonlinear state-update law and input matrix:

Combining Equations (16) and (17) gives the EKF prediction

$$\boxed{\bar{\mathbf{x}}[n+1] = g(\mathbf{x}[n], \mathbf{u}[n]) = \mathbf{f}(\mathbf{x}[n]) + B\mathbf{u}[n]}, \tag{19}$$

with

$$\mathbf{f}(\mathbf{x}[n]) = \begin{bmatrix} \theta[n] + \omega[n]\Delta t \\ x[n] + v[n]\Delta t \cos\theta[n] \\ y[n] + v[n]\Delta t \sin\theta[n] \\ a_v v[n] \\ a_\omega \omega[n] \end{bmatrix}, \qquad \boxed{B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ G_v(1-a_v) & 0 \\ 0 & G_\omega(1-a_\omega) \end{bmatrix}}. \tag{20}$$

The pose update is nonlinear because of the sine and cosine terms.

5. State Jacobian:

As in the Extended Kalman Filter lecture (slide 5), the state Jacobian is $G_x = \partial g/\partial \mathbf{x}$ (distinct from the gains G_v and G_ω). Differentiating the five rows of Equation (19) with respect to $[\theta, x, y, v, \omega]$ gives

$$\boxed{G_x = \begin{bmatrix} 1 & 0 & 0 & 0 & \Delta t \\ -v\Delta t \sin\theta & 1 & 0 & \Delta t \cos\theta & 0 \\ v\Delta t \cos\theta & 0 & 1 & \Delta t \sin\theta & 0 \\ 0 & 0 & 0 & a_v & 0 \\ 0 & 0 & 0 & 0 & a_\omega \end{bmatrix}}. \tag{21}$$

All state values in G_x are evaluated at the current state $\mathbf{x}[n]$. Because the command model is linear, $G_u = \partial g/\partial \mathbf{u} = B$. The EKF lecture's covariance prediction therefore takes the form

$$\bar{\Sigma}_x[n+1] = G_x \Sigma_x[n] G_x^T + B\Sigma_u B^T, \tag{22}$$

where $\Sigma_u = \text{diag}(\hat{\sigma}^2_{v_x}, \hat{\sigma}^2_{\omega_z}) = \text{diag}(0.00445^2,\ 0.00439^2)$ is the command noise measured in Assignment 1.

Assignment 5:

1. Wheel kinematics:

From the differential-drive equations provided in Lab 4 instructions (pages 4–5),

$$v[n] = \frac{r_w}{2}\big(\omega_r[n] + \omega_l[n]\big),$$
$$\omega[n] = \frac{r_w}{2R}\big(\omega_r[n] - \omega_l[n]\big), \tag{23}$$

where r_w is the wheel radius, R is half the wheel separation, and ω_r and ω_l are the right and left wheel angular velocities.

2. Measurements:

Rearranging Equation (23),

$$\omega_r[n] + \omega_l[n] = \frac{2v[n]}{r_w},$$
$$\omega_r[n] - \omega_l[n] = \frac{2R\omega[n]}{r_w}. \tag{24}$$

Adding and subtracting these expressions gives

$$\therefore \quad \omega_r[n] = \frac{v[n] + R\omega[n]}{r_w},$$
$$\omega_l[n] = \frac{v[n] - R\omega[n]}{r_w}. \tag{25}$$

The gyroscope measures body angular velocity directly, so $\omega_g[n] = \omega[n]$.

3. Measurement vector and C matrix:

Using the state vector from Assignment 4 and the measurement ordering specified in Lab 4,

$$\mathbf{x}[n] = \begin{bmatrix}\theta[n] & x[n] & y[n] & v[n] & \omega[n]\end{bmatrix}^T, \qquad \mathbf{z}[n] = \begin{bmatrix}\omega_r[n] & \omega_l[n] & \omega_g[n]\end{bmatrix}^T. \tag{26}$$

Equations (25) and $\omega_g[n] = \omega[n]$ produce the linear measurement model

$$\boxed{\mathbf{z}[n] = h(\mathbf{x}[n]) = C\mathbf{x}[n]}, \tag{27}$$

$$\boxed{C = \begin{bmatrix} 0 & 0 & 0 & 1/r_w & R/r_w \\ 0 & 0 & 0 & 1/r_w & -R/r_w \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}. \tag{28}$$

The first three columns are zero because the sensors do not directly measure θ, x, or y. Since h is linear, its measurement Jacobian is $H_x = \partial h/\partial \mathbf{x} = C$.

Given:

For the simulated TurtleBot 3 Waffle Pi, $r_w = 0.033\,\text{m}$ and its drive plugin uses a wheel separation of $2R = 0.287\,\text{m}$, so $R = 0.1435\,\text{m}$ (the 143.5 mm in the instructions is R). Spinning in place at $u_\omega = 0.4\,\text{rad/s}$ confirmed this: $\omega_r - \omega_l \approx 3.47\,\text{rad/s}$, against $2R\omega/r_w = 3.48\,\text{rad/s}$.

Results:

$$\frac{1}{r_w} = 30.30303\,\text{m}^{-1}, \qquad \frac{R}{r_w} = 4.34848, \tag{29}$$

$$\therefore \boxed{C \approx \begin{bmatrix} 0 & 0 & 0 & 30.30303 & 4.34848 \\ 0 & 0 & 0 & 30.30303 & -4.34848 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}}. \tag{30}$$

The wheel speeds come from `/joint_states` and the gyroscope measurement comes from `/imu`.

Measurement noise: The gyroscope variance is taken from the `/imu` message. `/joint_states` carries no covariance; since wheel speeds are a few to tens of rad/s, an encoder error of about 1% of a 10 rad/s reading is a reasonable guess: $\sigma_{enc} = 0.1\,\text{rad/s}$, i.e. a variance of $0.01\,(\text{rad/s})^2$. Thus $\Sigma_z = \text{diag}(0.01,\, 0.01,\, \sigma_g^2)$.

Assignment 6:

1. Prediction and correction:

The node `ekf_odom.py` implements the five-state model derived in Assignments 3–5, with $\mathbf{x} = [\theta, x, y, v, \omega]^T$, $\mathbf{u} = [u_v, u_\omega]^T$, and $\mathbf{z} = [\omega_r, \omega_l, \omega_g]^T$. Using the Extended Kalman Filter lecture (slides 5–7),

$$\begin{aligned}
\bar{\mathbf{x}}[n] &= g(\mathbf{x}[n-1], \mathbf{u}[n]), \\
\bar{\Sigma}_x[n] &= G_x \Sigma_x[n-1] G_x^T + B \Sigma_u B^T, \\
K[n] &= \bar{\Sigma}_x[n] C^T \left(C \bar{\Sigma}_x[n] C^T + \Sigma_z[n] \right)^{-1}, \\
\mathbf{x}[n] &= \bar{\mathbf{x}}[n] + K[n] \left(\mathbf{z}[n] - C \bar{\mathbf{x}}[n] \right), \\
\Sigma_x[n] &= (I - K[n] C) \bar{\Sigma}_x[n].
\end{aligned} \tag{31}$$

Here g, G_x, and B are from Assignment 4, and C is from Assignment 5. The elapsed time Δt is obtained from the sensor message timestamps rather than a fixed update period.

2. Measurements, synchronization, and covariance:

The node subscribes to `/cmd_vel` (`TwistStamped`), `/imu` (`Imu`), and `/joint_states` (`JointState`). The IMU and wheel measurements are paired using `ApproximateTimeSynchronizer`. In simulation time, `/imu` is published at 200 Hz (5 ms) and `/joint_states` at 500 Hz (2 ms, every physics step), measured as 185 and 463 Hz of wall-clock time with `ros2 topic hz`. Every IMU sample therefore has a wheel sample within about 1 ms, so the slop was set to 0.01 s: wide enough to tolerate jitter, yet well below the 37 ms velocity rise time. During the evaluation run the node fused about 183 pairs per second of wall-clock time, essentially every IMU sample, with a worst IMU/encoder offset of 6 ms. Wheel velocities are selected by joint name, and the angular velocity measurement uses the gyroscope's z component.

Commands older than 1 s are treated as zero, following the robot's command timeout. Each command is time-stamped on arrival with the latest sensor timestamp, so the timeout is measured in simulation time; the command header is not used because it is zero from `ros2 topic pub` and wall-clock time from `rqt_robot_steering`.

The input covariance Σ_u uses the measured zero-command velocity variances from Assignment 1. The encoder standard deviation is 0.1 rad/s (Assignment 5). The gyroscope z variance is read from the IMU message, which reports $\sigma_g^2 = 1.6 \times 10^{-7}\,(\text{rad/s})^2$; 0.01^2 is used only as a fallback if no variance is reported. No external pose or ground-truth measurements enter the filter.

3. Odometry output:

The node publishes `nav_msgs/msg/Odometry` on `/ekf_odom`, using `odom` as the pose frame and `base_footprint` as the body frame. The local initial pose is $(\theta, x, y) = (0, 0, 0)$. The estimated heading θ is converted to a quaternion with $q_z = \sin(\theta/2)$ and $q_w = \cos(\theta/2)$. The 5×5 covariance Σ_x is mapped into the appropriate entries of the two 6×6 ROS pose and twist covariance matrices. The output timestamp is taken from the synchronized sensor readings.

Code: `prob_rob_labs/src/lab4_odom/ekf_odom.py` (registered as `ekf_odom` in `setup.py`); full listing in Appendix A.

Assignment 7:

1. Setup:

The EKF node was started before the robot was first commanded, so its `odom` frame is the robot's initial pose. The robot spawns at the `world` origin with zero yaw (`turtlebot3_empty_launch.py`), so the odometry-to-world transform is the identity and the estimate is compared with ground truth directly:

$$T_b^w = T_o^w \, T_b^o = T_b^o, \qquad T_o^w = I. \tag{32}$$

T_o^w was published with the identity static transform from Assignment 2, and RViz was run with Fixed Frame = `world`, displaying `/tb3/ground_truth/pose` (red arrow) and `/ekf_odom` (blue arrows). The robot was driven by `drive_pattern.py` through a 71 s pattern with $u_v = 0.15$ m/s while driving: parked 5 s, straight 10 s, left arc ($u_\omega = 0.4$ rad/s) 10 s, straight 10 s, right arc ($u_\omega = -0.4$ rad/s) 10 s, spin in place ($u_\omega = 0.5$ rad/s) 6 s, straight 10 s, and parked 10 s.

2. Result:

Yes. Throughout the drive, the EKF pose (blue) stays on the ground-truth pose (red) in RViz (video and Figure 5). The largest discrepancy, measured in Assignment 8, is about 5 cm and 0.6° after 7.5 m of driving.

Video: EKF odometry and ground truth in RViz (also `assignment7/assignment7_ekf_vs_ground_truth.mp4` in the submission archive).

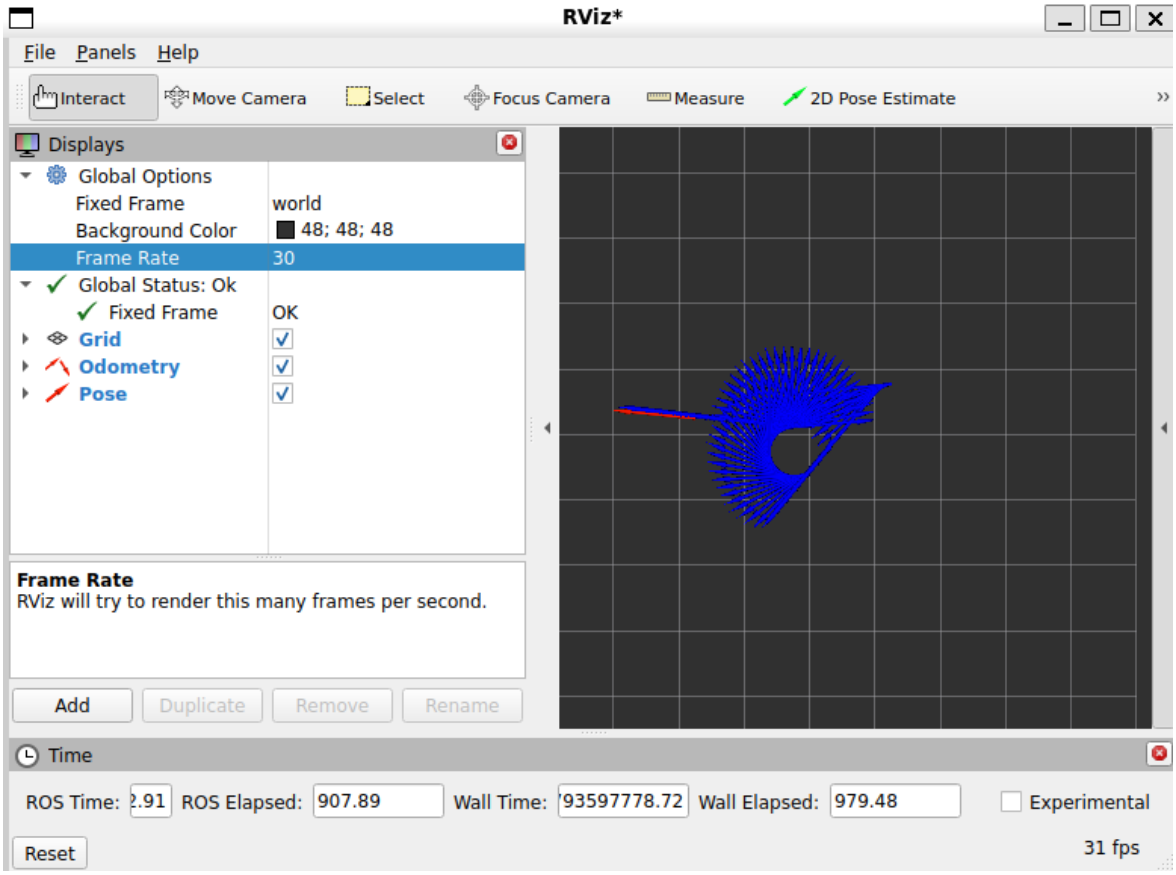


Figure 5: RViz (Fixed Frame `world`) at the end of the run: the ground-truth pose (red) lies on the EKF odometry trail (blue).

Assignment 8:

1. Error node:

The node `ekf_error.py` (registered as `ekf_error`; listing in Appendix B) subscribes to `/tb3/ground_truth/pose` (`PoseStamped`, `world` frame) and `/ekf_odom` (`Odometry`, `odom` frame). Both are stamped in simulation time, so they are paired with `ApproximateTimeSynchronizer` (slop = 0.01 s; `/ekf_odom` runs at about 200 Hz, so every ground-truth pose has an estimate within about 2.5 ms). Because $T_o^w = I$ (Assignment 7), the paired poses are compared directly.

2. Error metrics:

The Euclidean position error is

$$\boxed{e_p[n] = \sqrt{(\hat{x}[n] - x_{gt}[n])^2 + (\hat{y}[n] - y_{gt}[n])^2}}. \tag{33}$$

The heading of each pose is extracted from its quaternion,

$$\theta = \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\big), \tag{34}$$

and the heading difference is wrapped to $[-\pi, \pi]$, so that, for example, 179° and −179° differ by 2° rather than 358°:

$$\boxed{e_\theta[n] = \frac{180}{\pi}\,\big|\operatorname{atan2}\big(\sin\Delta\theta[n],\ \cos\Delta\theta[n]\big)\big|}, \qquad \Delta\theta[n] = \hat{\theta}[n] - \theta_{gt}[n]. \tag{35}$$

The errors are published as `std_msgs/msg/Float64` on `/ekf_error/position` (m) and `/ekf_error/yaw` (deg), and were plotted in PlotJuggler during the Assignment 7 run.

3. Results:

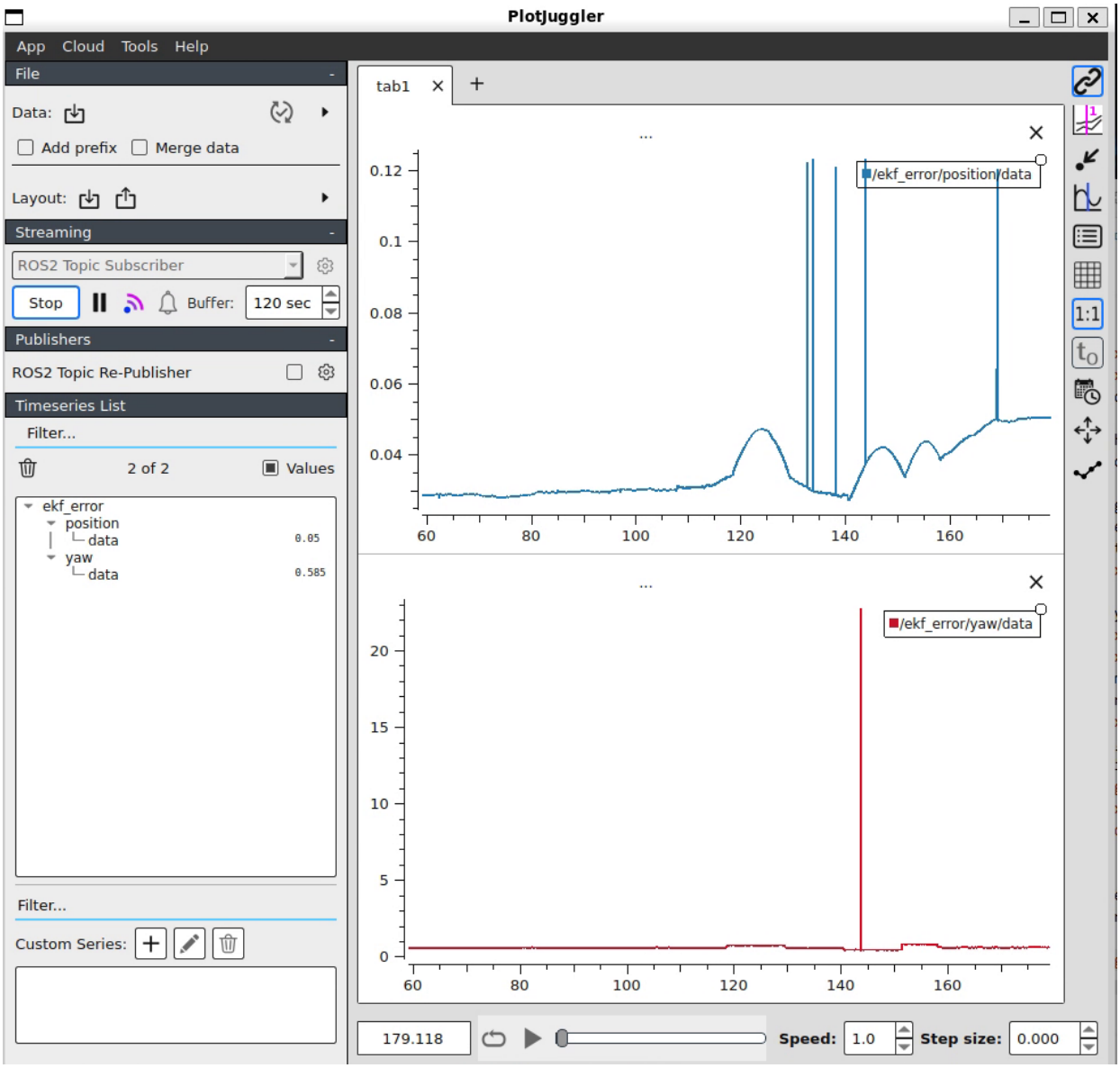


Figure 6: Position error e_p and heading error e_θ during the 71 s drive pattern, plotted in PlotJuggler.

4. Observations:

Over the 71 s pattern (7.5 m driven), e_p rose from 3.0 to 5.0 cm and e_θ stayed between 0.37° and 0.87°, apart from the isolated spikes discussed below. Most of this error was present before the robot moved: the 0.58° heading offset existed before the drive and stayed nearly constant over the whole 6.7-minute recording, so it is a small misalignment between `odom` and `world` (most likely the robot rotated slightly under the injected command noise before the EKF was started), not drift. A fixed rotation offset produces a position error proportional to the distance from where the EKF started (0.6° ≈ 1 cm per metre), which is why e_p rises and falls during the arcs. Removing a best-fit rigid transform (rotation 0.69°) leaves a residual of at most 1.2 cm over the whole drive, so the filter itself drifted by about 1 cm over 7.5 m. The heading did not drift measurably because the gyroscope measures ω with $\sigma_g = 4 \times 10^{-4}$ rad/s.

The isolated spikes (5 of 4,446 samples during the drive) are not estimation errors: the matching ground-truth messages carry timestamps that are off by exactly 1 s (38 of 21,738 messages in the recording, a seconds/nanoseconds glitch in `gz_pose_publisher`), so the synchronizer pairs them with an estimate from 1 s away.

Assignment 9:

1. Published covariance:

The 5×5 state covariance Σ_x is published in the two 6×6 `Odometry` covariances (row-major, ordered $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$

and $[v_x, v_y, v_z, \omega_x, \omega_y, \omega_z]$): Σ_{xx}, Σ_{yy}, $\Sigma_{\theta\theta}$ at pose indices 0, 7, 35 (cross terms at 1/6, 5/30, 11/31), and Σ_{vv}, $\Sigma_{\omega\omega}$ at twist indices 0 and 35 (cross term at 5/30). All other entries are zero. Figure 7 shows the standard deviations (square roots of the diagonal) over the Assignment 7 run, from the recorded `/ekf_odom`.

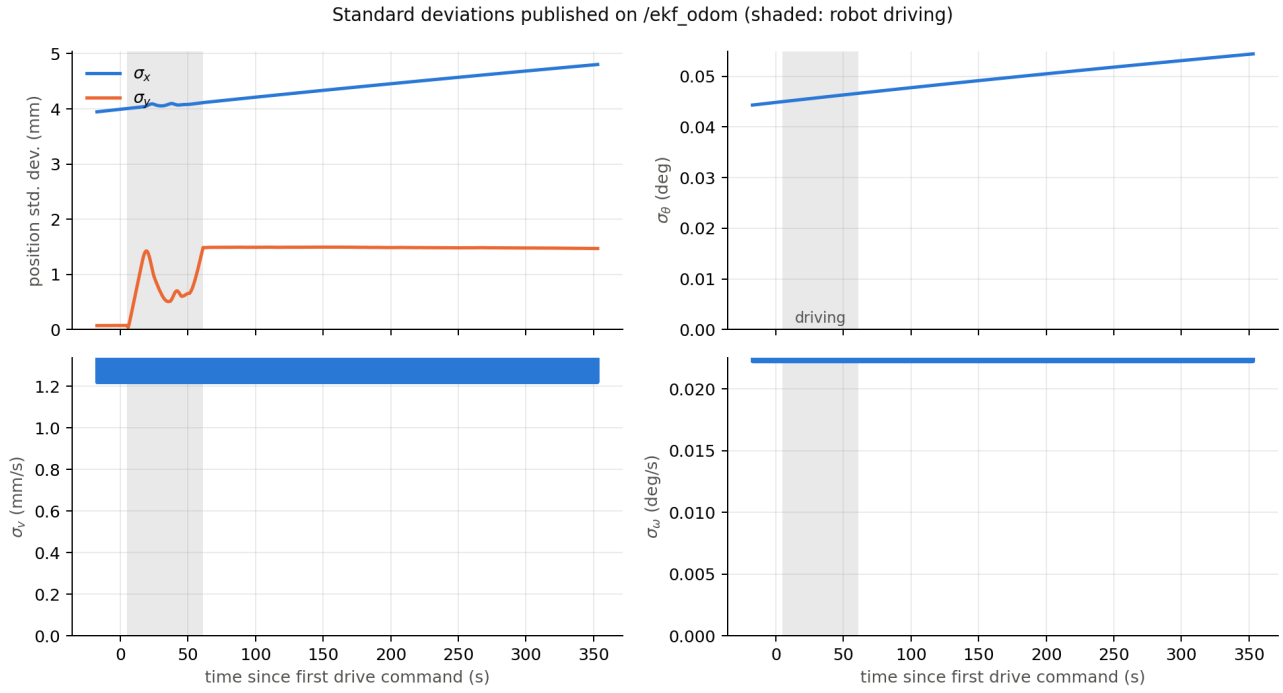


Figure 7: Standard deviations (square roots of the diagonal of Σ_x) published on `/ekf_odom` during the evaluation recording; shaded: robot driving. Generated by `analyze_eval_run.py`.

2. Stable: Σ_{vv} and $\Sigma_{\omega\omega}$. These are constant over the whole recording, $\sigma_v = 1.2$–$1.3\,\text{mm/s}$ and $\sigma_\omega = 0.022°/\text{s}$, whether the robot is moving or parked. The last two columns of C are nonzero, so v and ω are measured at every update: each correction removes the uncertainty that the prediction adds through $B\Sigma_u B^T$, and the two variances converge to a steady state.

3. Growing without bound: $\Sigma_{\theta\theta}$, Σ_{xx}, Σ_{yy}. The first three columns of C are zero: no sensor observes θ, x, or y, so nothing anchors them. Every prediction adds uncertainty through G_x, and the corrections affect them only through their correlations with v and ω. From the first row of G_x,

$$\bar{\Sigma}_{\theta\theta}[n+1] = \Sigma_{\theta\theta}[n] + 2\Delta t\,\Sigma_{\theta\omega}[n] + \Delta t^2\,\Sigma_{\omega\omega}[n], \tag{36}$$

so the heading variance accumulates the bounded ω uncertainty at every step, like a random walk: σ_θ grows steadily from 0.044° to 0.054° over the recording, moving or parked. The position rows add v uncertainty along the heading $(\Delta t\cos\theta,\ \Delta t\sin\theta)$ and heading uncertainty across it $(-v\Delta t\sin\theta,\ v\Delta t\cos\theta)$. While the robot drives, both act, and $\Sigma_{xx}+\Sigma_{yy}$ grows about 2.6 times faster than while parked; σ_y rises from 0.07 to 1.5 mm during the drive (dipping during the arcs, where errors from driving out and back partly cancel). When the robot is parked, only the along-heading term through Σ_{vv} remains: it was parked facing $\pm x$ before and after the drive, so σ_x kept growing (3.9 to 4.8 mm) while σ_y stayed flat.

4. Covariance versus actual error: At the end of the drive, $\sqrt{\Sigma_{xx}+\Sigma_{yy}} = 4.4\,\text{mm}$ and $\sigma_\theta = 0.047°$, while the measured errors were 50 mm and 0.59° (Assignment 8), 11 and 13 times larger. Most of that gap is the `odom`/`world` misalignment, which lies outside the filter's state (its pose covariance starts at zero because `odom` is defined at the start pose). Even the residual drift after removing it, up to 12 mm, is about 2–3 times the reported σ, so the filter is overconfident: Σ_u models only the command noise, not wheel slip or model mismatch.

Odometry alone therefore cannot bound the pose uncertainty; that requires an external reference such as landmarks.

Appendix A: ekf_odom.py

```
#!/usr/bin/env python3
"""Lab 4: estimate planar odometry using wheel encoders and the IMU."""

import math

import message_filters
import numpy as np
import rclpy
from geometry_msgs.msg import TwistStamped
from nav_msgs.msg import Odometry
from rclpy.node import Node
from sensor_msgs.msg import Imu, JointState


class EkfOdom(Node):
    """Track [theta, x, y, v, omega] using the Lab 4 EKF equations."""

    def __init__(self):
        """Initialize the state, covariances, and ROS interfaces."""
        super().__init__('ekf_odom')

        # State [theta, x, y, v, omega] and 5x5 covariance.
        self.x = np.zeros((5, 1))
        self.Sigma_x = np.diag([0.0, 0.0, 0.0, 0.01, 0.01])
        self.last_stamp = None
        self.cmd_stamp = None
        self.u = np.zeros((2, 1)) # [u_v, u_omega]

        # Values from assignment 3
        self.G_v = 0.9965
        self.G_omega = 1.0085
        self.tau_v = 0.0367
        self.tau_omega = 0.0212

        # Input noise approximated from assignment 1 zero-command data
        self.Sigma_u = np.diag([0.004448870**2, 0.004391655**2])

        # assignment 5 wheel geometry and measurement model
        r_w = 0.033
        R = 0.287 / 2.0 # waffle_pi DiffDrive wheel_separation is 0.287 m
        self.C = np.array([
            [0, 0, 0, 1 / r_w, R / r_w],
            [0, 0, 0, 1 / r_w, -R / r_w],
            [0, 0, 0, 0, 1],
        ])
        self.encoder_var = 0.1**2 # Assumed encoder standard deviation: 0.1 rad/s
        self.gyro_var = 0.01**2 # Fallback if the IMU has no valid variance
        self.cmd_timeout = 1.0 # s, same as the robot's command safety timer
        self.missing_wheels_logged = False
        self.sync_count = 0
        self.max_sync_offset = 0.0

        self.cmd_sub = self.create_subscription(
            TwistStamped, '/cmd_vel', self.handle_command, 10)
        self.imu_sub = message_filters.Subscriber(self, Imu, '/imu')
        self.joint_sub = message_filters.Subscriber(self, JointState, '/joint_states')
        # /imu is 200 Hz (5 ms) and /joint_states 500 Hz (2 ms) in sim time,
        # so every IMU sample has an encoder sample within ~1 ms. 10 ms
        # tolerates jitter while keeping pairs well inside the 37 ms rise time.
        self.sync = message_filters.ApproximateTimeSynchronizer(
            [self.imu_sub, self.joint_sub], queue_size=20, slop=0.01)
        self.sync.registerCallback(self.handle_measurements)
        self.odom_pub = self.create_publisher(Odometry, '/ekf_odom', 10)

    def handle_command(self, msg):
        """Store the commanded body velocities and their arrival time.

        The arrival time is the latest processed sensor timestamp, so the
        timeout is measured in sensor (simulation) time. The command header is
        not used: it is zero from `ros2 topic pub` and wall-clock time from
        rqt_robot_steering.

        Args:
            msg: Commanded velocity (TwistStamped).
        """
        self.u[:, 0] = [msg.twist.linear.x, msg.twist.angular.z]
        self.cmd_stamp = self.last_stamp

    def predict(self, dt, u):
        """Predict the state and covariance using Assignments 3 and 4.

        Args:
            dt: Sensor-to-sensor elapsed simulation time in seconds.
            u: Current [u_v, u_omega] command (2x1).
```

```
        """
        theta, px, py, v, omega = self.x[:, 0]
        a_v = 0.1 ** (dt / self.tau_v)
        a_omega = 0.1 ** (dt / self.tau_omega)

        # Nonlinear state update from Assignment 4
        f = np.array([
            [theta + omega * dt],
            [px + v * dt * math.cos(theta)],
            [py + v * dt * math.sin(theta)],
            [a_v * v],
            [a_omega * omega],
        ])
        B = np.array([
            [0, 0], [0, 0], [0, 0],
            [self.G_v * (1 - a_v), 0],
            [0, self.G_omega * (1 - a_omega)],
        ])
        G_x = np.array([
            [1, 0, 0, 0, dt],
            [-v * dt * math.sin(theta), 1, 0, dt * math.cos(theta), 0],
            [v * dt * math.cos(theta), 0, 1, dt * math.sin(theta), 0],
            [0, 0, 0, a_v, 0],
            [0, 0, 0, 0, a_omega],
        ])

        self.x = f + B @ u
        self.Sigma_x = (G_x @ self.Sigma_x @ G_x.T
                        + B @ self.Sigma_u @ B.T)

    def correct(self, z, gyro_var):
        """Correct the predicted state using wheel and gyroscope readings.

        Args:
            z: [omega_r, omega_l, omega_g] measurement column (3x1).
            gyro_var: Gyroscope z-axis measurement variance.
        """
        Sigma_z = np.diag([self.encoder_var, self.encoder_var, gyro_var])
        S = self.C @ self.Sigma_x @ self.C.T + Sigma_z
        K = self.Sigma_x @ self.C.T @ np.linalg.inv(S)

        # Measurement model h(x)=Cx and standard EKF innovation
        self.x = self.x + K @ (z - self.C @ self.x)
        self.Sigma_x = (np.eye(5) - K @ self.C) @ self.Sigma_x
        self.Sigma_x = (self.Sigma_x + self.Sigma_x.T) / 2.0

    def handle_measurements(self, imu, joints):
        """Process time-aligned sensor readings and publish an odometry estimate.

        Args:
            imu: IMU message containing gyroscope z reading and variance.
            joints: JointState message containing left/right wheel velocities.
        """
        # JointState entries must be identified by name, not array order
        if 'wheel_right_joint' not in joints.name or 'wheel_left_joint' not in joints.name:
            if not self.missing_wheels_logged:
                self.get_logger().warning('Wheel joints not found in /joint_states.')
                self.missing_wheels_logged = True
            return

        right = joints.name.index('wheel_right_joint')
        left = joints.name.index('wheel_left_joint')
        if len(joints.velocity) <= max(right, left):
            return

        # Use the later of the two synchronized measurement timestamps
        imu_time = imu.header.stamp.sec + imu.header.stamp.nanosec * 1e-9
        joint_time = joints.header.stamp.sec + joints.header.stamp.nanosec * 1e-9
        stamp = max(imu_time, joint_time)
        if stamp <= 0.0 or (self.last_stamp is not None and stamp <= self.last_stamp):
            return
        self.log_sync(abs(imu_time - joint_time))

        # Treat commands older than the robot's timeout as zero velocity
        age = None if self.cmd_stamp is None else stamp - self.cmd_stamp
        u = self.u if age is not None and age < self.cmd_timeout else np.zeros((2, 1))
        if self.last_stamp is not None:
            self.predict(stamp - self.last_stamp, u)

        z = np.array([[joints.velocity[right]],
                      [joints.velocity[left]],
                      [imu.angular_velocity.z]])

        # IMU supplies covariance, but use fallback if unknown
        gyro_var = imu.angular_velocity_covariance[8]
        if gyro_var <= 0.0:
            gyro_var = self.gyro_var
```

```
        self.correct(z, gyro_var)
        self.last_stamp = stamp
        self.publish_odometry(imu.header.stamp if imu_time >= joint_time
                              else joints.header.stamp)

    def log_sync(self, offset):
        """Log␣every␣5␣s,␣how␣many␣pairs␣were␣fused␣and␣their␣worst␣stamp␣offset.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣offset:␣|IMU␣stamp␣-␣encoder␣stamp|␣of␣the␣current␣pair␣in␣seconds.
␣␣␣␣␣␣␣␣"""
        self.sync_count += 1
        self.max_sync_offset = max(self.max_sync_offset, offset)
        self.get_logger().info(
            f'{self.sync_count}␣synchronized␣pairs,␣max␣IMU/encoder␣offset␣'
            f'{self.max_sync_offset␣*␣1e3:.1f}␣ms', throttle_duration_sec=5.0)

    def publish_odometry(self, stamp):
        """Publish␣the␣state␣and␣covariance␣in␣a␣ROS␣Odometry␣message.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣stamp:␣Timestamp␣of␣the␣latest␣synchronized␣sensor␣measurement.
␣␣␣␣␣␣␣␣"""
        theta, px, py, v, omega = self.x[:, 0]
        odom = Odometry()
        odom.header.stamp = stamp
        odom.header.frame_id = 'odom'
        odom.child_frame_id = 'base_footprint'
        odom.pose.pose.position.x = float(px)
        odom.pose.pose.position.y = float(py)
        odom.pose.pose.orientation.z = math.sin(theta / 2.0)
        odom.pose.pose.orientation.w = math.cos(theta / 2.0)
        odom.twist.twist.linear.x = float(v)
        odom.twist.twist.angular.z = float(omega)

        # ROS pose order: [x, y, z, roll, pitch, yaw].
        pose_indices = [0, 1, 5] # ROS covariance indices for x, y, yaw.
        state_pose_indices = [1, 2, 0]
        for i, pi in enumerate(pose_indices):
            for j, pj in enumerate(pose_indices):
                odom.pose.covariance[6 * pi + pj] = float(
                    self.Sigma_x[state_pose_indices[i], state_pose_indices[j]])

        # ROS twist order: [linear x, y, z, angular x, y, z].
        odom.twist.covariance[0] = float(self.Sigma_x[3, 3])
        odom.twist.covariance[5] = float(self.Sigma_x[3, 4])
        odom.twist.covariance[30] = float(self.Sigma_x[4, 3])
        odom.twist.covariance[35] = float(self.Sigma_x[4, 4])
        self.odom_pub.publish(odom)


def main():
    """Run␣the␣EKF␣node␣until␣interrupted."""
    rclpy.init()
    node = EkfOdom()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Appendix B: `ekf_error.py`

```
#!/usr/bin/env python3
"""Lab␣4,␣Assignment␣8:␣error␣between␣the␣EKF␣odometry␣and␣ground␣truth."""

import math

import message_filters
import rclpy
from geometry_msgs.msg import PoseStamped
from nav_msgs.msg import Odometry
from rclpy.node import Node
from std_msgs.msg import Float64


def yaw_from_quaternion(q):
```

```
    """Return␣the␣rotation␣about␣z␣(rad)␣of␣a␣quaternion."""
    return math.atan2(2.0 * (q.w * q.z + q.x * q.y),
                      1.0 - 2.0 * (q.y * q.y + q.z * q.z))


def wrap_angle(angle):
    """Wrap␣an␣angle␣to␣[-pi,␣pi]␣so␣errors␣across␣+/-180␣deg␣stay␣small."""
    return math.atan2(math.sin(angle), math.cos(angle))


class EkfError(Node):
    """Publish␣position␣and␣heading␣error␣of␣/ekf_odom␣against␣ground␣truth.

␣␣␣␣The␣odom␣frame␣coincides␣with␣world␣because␣the␣robot␣spawns␣at␣the
␣␣␣␣world␣origin␣with␣zero␣yaw␣and␣the␣EKF␣is␣started␣before␣the␣robot␣moves.
␣␣␣␣"""

    def __init__(self):
        super().__init__('ekf_error')
        self.truth_sub = message_filters.Subscriber(
            self, PoseStamped, '/tb3/ground_truth/pose')
        self.odom_sub = message_filters.Subscriber(self, Odometry, '/ekf_odom')
        # Both topics carry simulation-time stamps from the sensors/simulator.
        # /ekf_odom runs at ~200 Hz, so any ground-truth pose has an estimate
        # within ~2.5 ms. The queue holds 1 s of estimates because ground
        # truth arrives through the gz CLI with extra latency.
        self.sync = message_filters.ApproximateTimeSynchronizer(
            [self.truth_sub, self.odom_sub], queue_size=200, slop=0.01)
        self.sync.registerCallback(self.handle_pair)
        self.position_pub = self.create_publisher(Float64, '/ekf_error/position', 10)
        self.yaw_pub = self.create_publisher(Float64, '/ekf_error/yaw', 10)

    def handle_pair(self, truth, odom):
        """Compute␣and␣publish␣the␣errors␣for␣one␣time-aligned␣pair.

␣␣␣␣␣␣␣␣Args:
␣␣␣␣␣␣␣␣␣␣␣␣truth:␣Ground-truth␣pose␣(PoseStamped,␣world␣frame).
␣␣␣␣␣␣␣␣␣␣␣␣odom:␣EKF␣estimate␣(Odometry,␣odom␣frame).
␣␣␣␣␣␣␣␣"""
        est = odom.pose.pose
        dx = est.position.x - truth.pose.position.x
        dy = est.position.y - truth.pose.position.y
        dyaw = wrap_angle(yaw_from_quaternion(est.orientation)
                          - yaw_from_quaternion(truth.pose.orientation))

        self.position_pub.publish(Float64(data=math.hypot(dx, dy)))
        self.yaw_pub.publish(Float64(data=abs(math.degrees(dyaw))))


def main():
    """Run␣the␣error␣node␣until␣interrupted."""
    rclpy.init()
    node = EkfError()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```